

chapter - 05

Micro-Sequencer Control Unit Design

5.1. Micro-sequencer operations and Microinstruction format

#. Micro-operations

↳ Programs are executed as a sequence of instructions. Each instruction consists of a series of steps that make the instruction cycle fetch-decode (indirect), execute and interrupt with fetch and execute cycles always occurring. For designing the control unit, we need to break down the ~~dis~~ designing the control unit description further. Each of the smaller cycles involves a series of steps, each of which involves the processor registers.

The smallest elementary functional operations performed within the computer are called micro-operations.

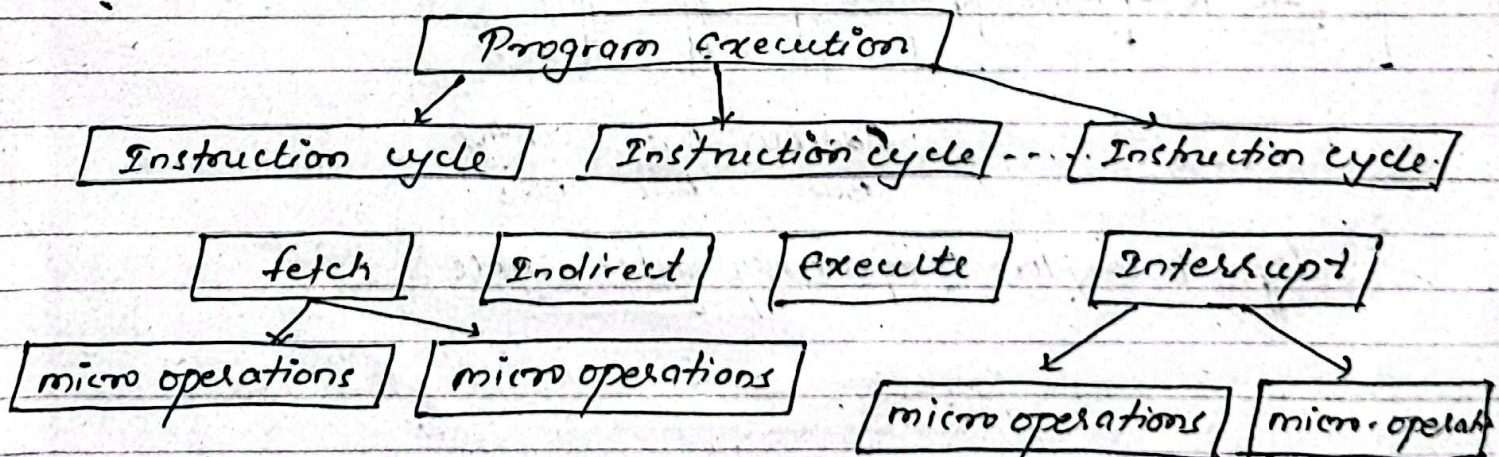


fig: constituent elements of program execution.

• Fetch cycle

$t_1: MAR \leftarrow PC$

$t_2: MBR \leftarrow Memory, PC \leftarrow PC + 1$

$t_3: IR \leftarrow MBR$

where,

MBR = Memory Buffer Register

MAR = Memory Address Register

IR = Instruction Register

PC = Program Counter

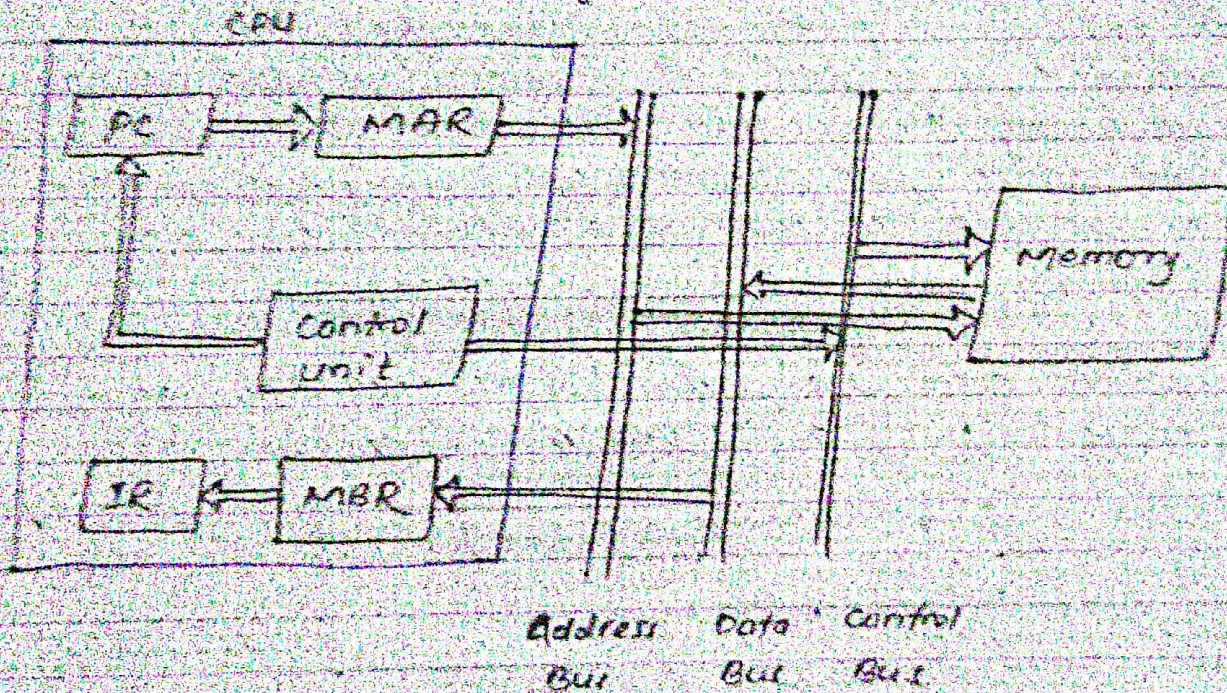


Fig. Data flow diagram of fetch cycle.

b. Indirect cycle.

$t_1 \Rightarrow \text{MAR} \leftarrow (\text{IR address})$

$t_2 \Rightarrow \text{MBR} \leftarrow \text{Memory}$

$t_3 \Rightarrow \text{IR (address)} \leftarrow \text{MBR (address)}$

used to fetch operand.

c. Execute cycle.

→ to execute micro-instructions

→ different for different instructions

eg: $\text{ADD } R_1, X$

$t_1 \Rightarrow \text{MAR} \leftarrow (\text{IR address})$

$t_2 \Rightarrow \text{MBR} \leftarrow \text{Memory}$

$t_3 \Rightarrow R_1 \leftarrow (R_1) + \text{MBR}$

d. Interrupt cycle:

$t_1 \Rightarrow \text{MBR} \leftarrow (\text{PC})$

$t_2 \Rightarrow \text{MAR} \leftarrow \text{save-address}$

$\text{PC} \leftarrow \text{Routine address}$

$t_3 \Rightarrow \text{Memory} \leftarrow \text{MBR}$

Microinstructions

In microprogrammed control unit in addition to use of control signals, each microoperations is described in symbolic notation. This notation looks like programming language called micro programming.

Each lines of microprogramming describes set of microoperations occurring at a time called micro-instructions. The memory where microinstructions are stored is called control memory.

There are 2 types of microinstructions:

- a) Horizontal microinstruction } typical micro-instruction
- b) Vertical microinstruction. } formats.

a) Horizontal microinstruction:

When the length of microinstruction is more than 40 bit but less than 100 bit then it is called horizontal microinstruction. Each micro operation occurring at a same time is represented by a bit value (1 if occurs and 0 if not).

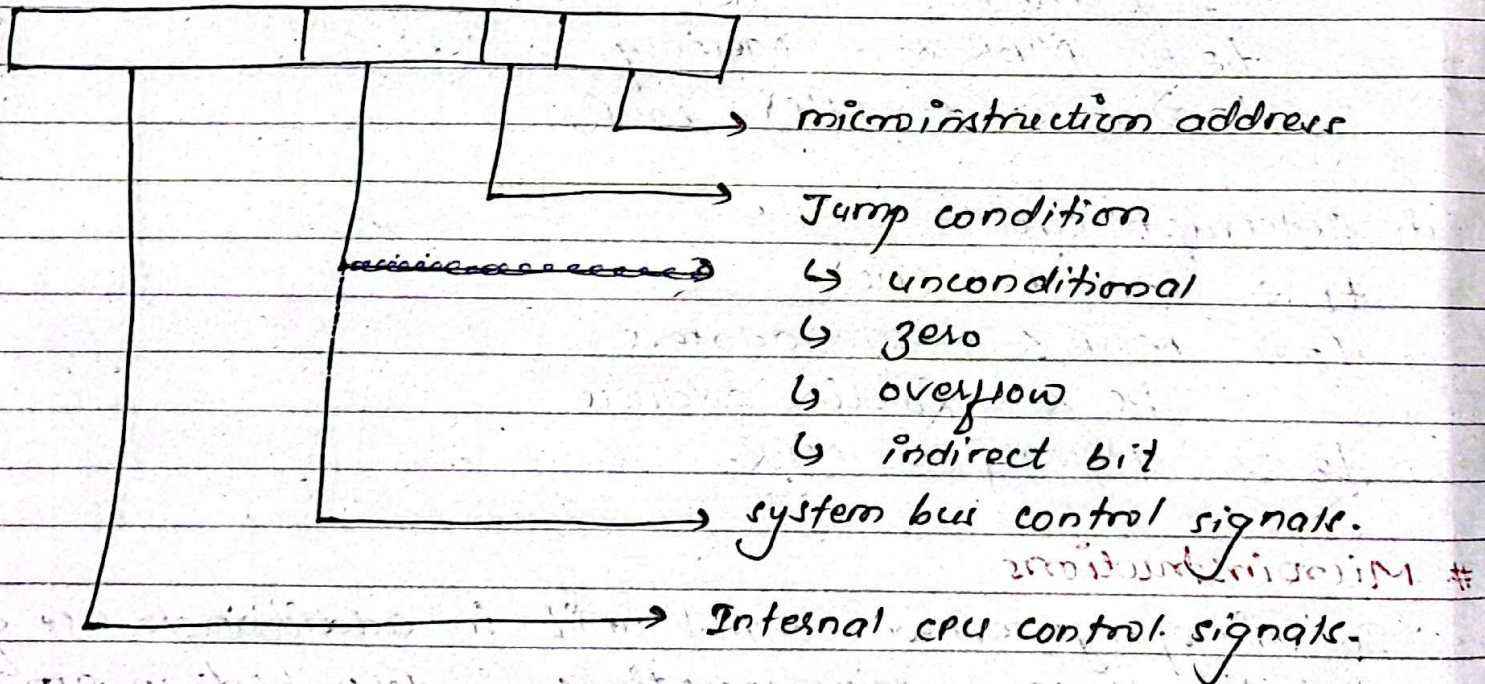


fig: Horizontal microinstructions.

During the execution of an instruction, the control unit fetches the appropriate microinstruction from the control store and decodes the control signals to control the operation of the processor.

The advantage of horizontal microinstructions is that they allow for flexible control over the processor's operations. By representing control signals horizontally, it is easier to modify or add new control signals without disrupting the entire microinstruction word.

2. Vertical microinstruction.

If the length of microinstruction is more than 16 bit but less than 40 bit it is called vertical microinstruction. Group of bits represent unique set of microoperation occurring at a time.

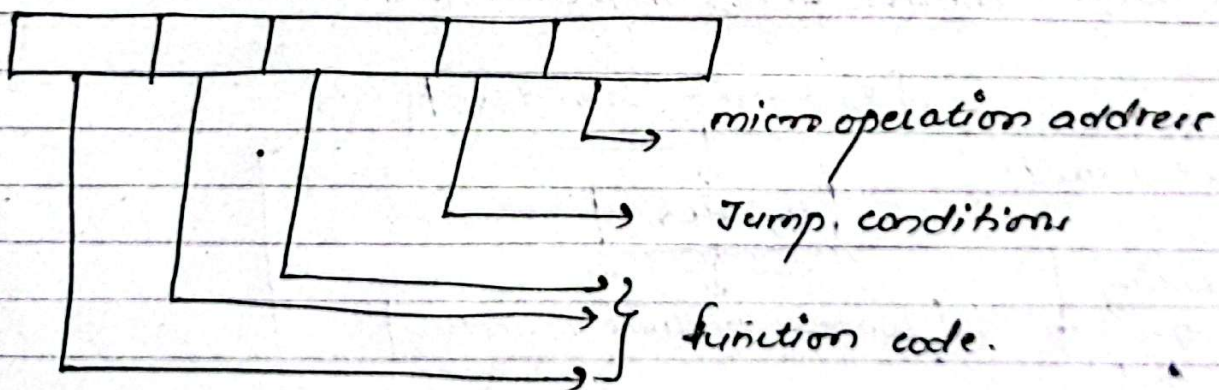


Fig: vertical microinstruction.

The advantage of vertical microinstructions is their compactness and simplicity. Since the control signals are represented vertically, the microinstruction word can be narrower compared to horizontal microinstructions. This results in more efficient memory utilization and potentially faster access times. However, one drawback of vertical microinstruction is that modifying or adding new control signals may require changes to the entire microinstruction word.

Instructions:

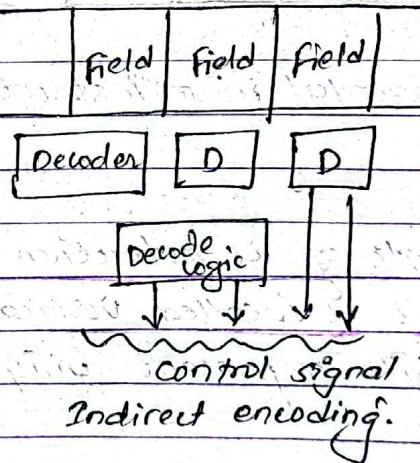
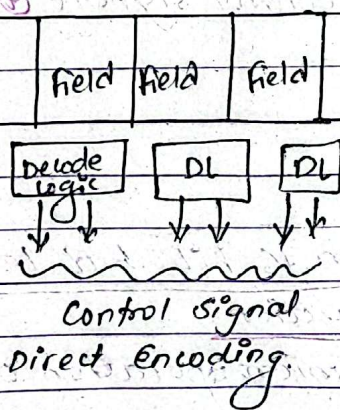
Format of microinstruction

Horizontal / vertical

Packed / unpacked: → It is related to degree of packing. If more no. of bits define microoperation then called packed. Otherwise unpacked. Packed microinstruction holds more information.

Hard and Soft: It suggests degree of closeness to underlining control signals and hardware layout. Hard microinstructions are fixed whereas software changeable.

Direct / Indirect Encoding: If only one layer decoding logic is used direct if new multilayer decoding logic used indirect.



4 Hardwired Implementation of CU. (2016 sp)

In this approach control signals are defined by control functions expressed in terms of control input line status of flag, control of IR, control signal from Bus etc. The control signal

5.2. Design and implementation of a very simple micro sequences.

A microsequencer, also known as a control sequencer, is a component in a computer architecture that generates a sequence of control signals to execute instructions. A simple microsequencer consists of a counter, a control store (microinstruction memory) and a decoder.

For microinstruction sequencing, a dedicated hardware sequential logic unit is employed. The sequencer generates the address of next microinstructions are radially stored in the control memory. Thus, microinstructions executed in a sequence thoroughly generates the required control signals. This approach is flexible than hardware control unit implementation.

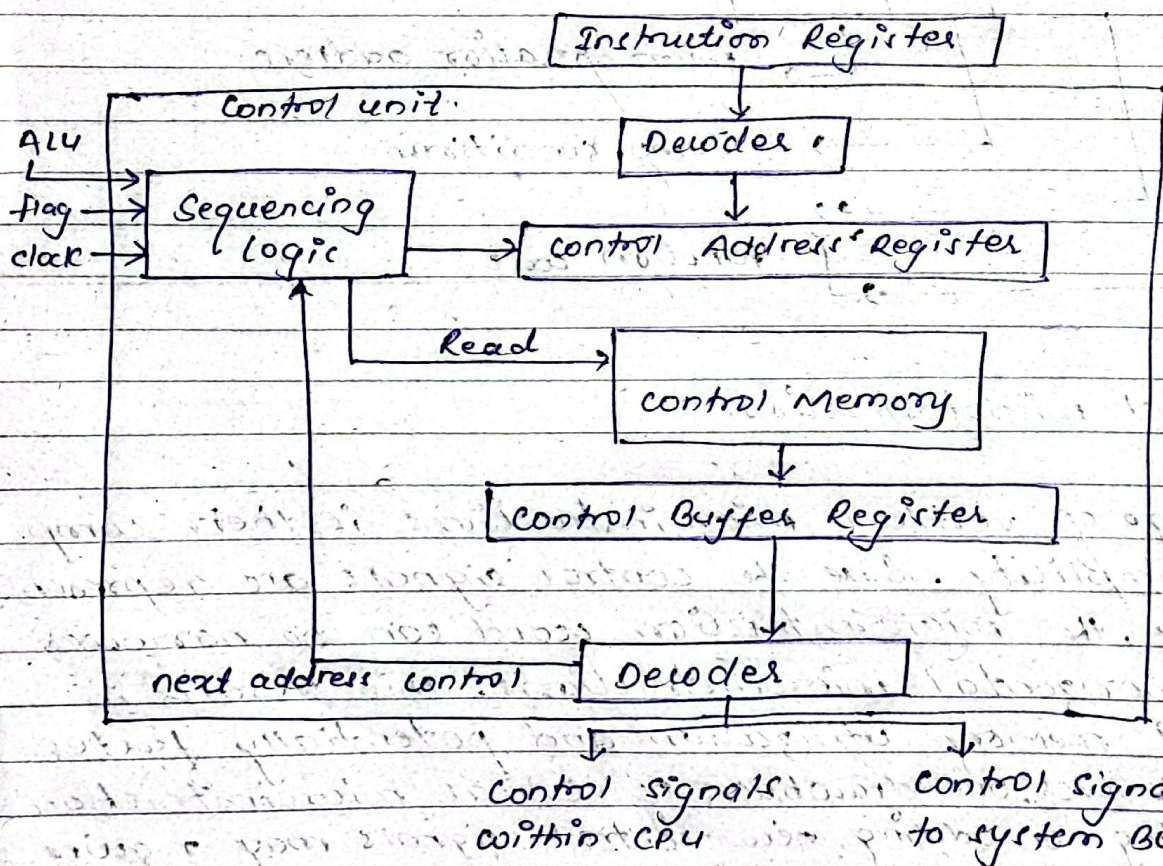


fig: layout of microsequencer.

#. Sequencing Techniques.

The current instruction determines the address of next instruction on the basis of status bit condition of flag and next address captured by current microinstructions. Based on the format of address, microinstruction sequencing techniques are different.

- two address field
- single address format
- variable address format.

as two address field.

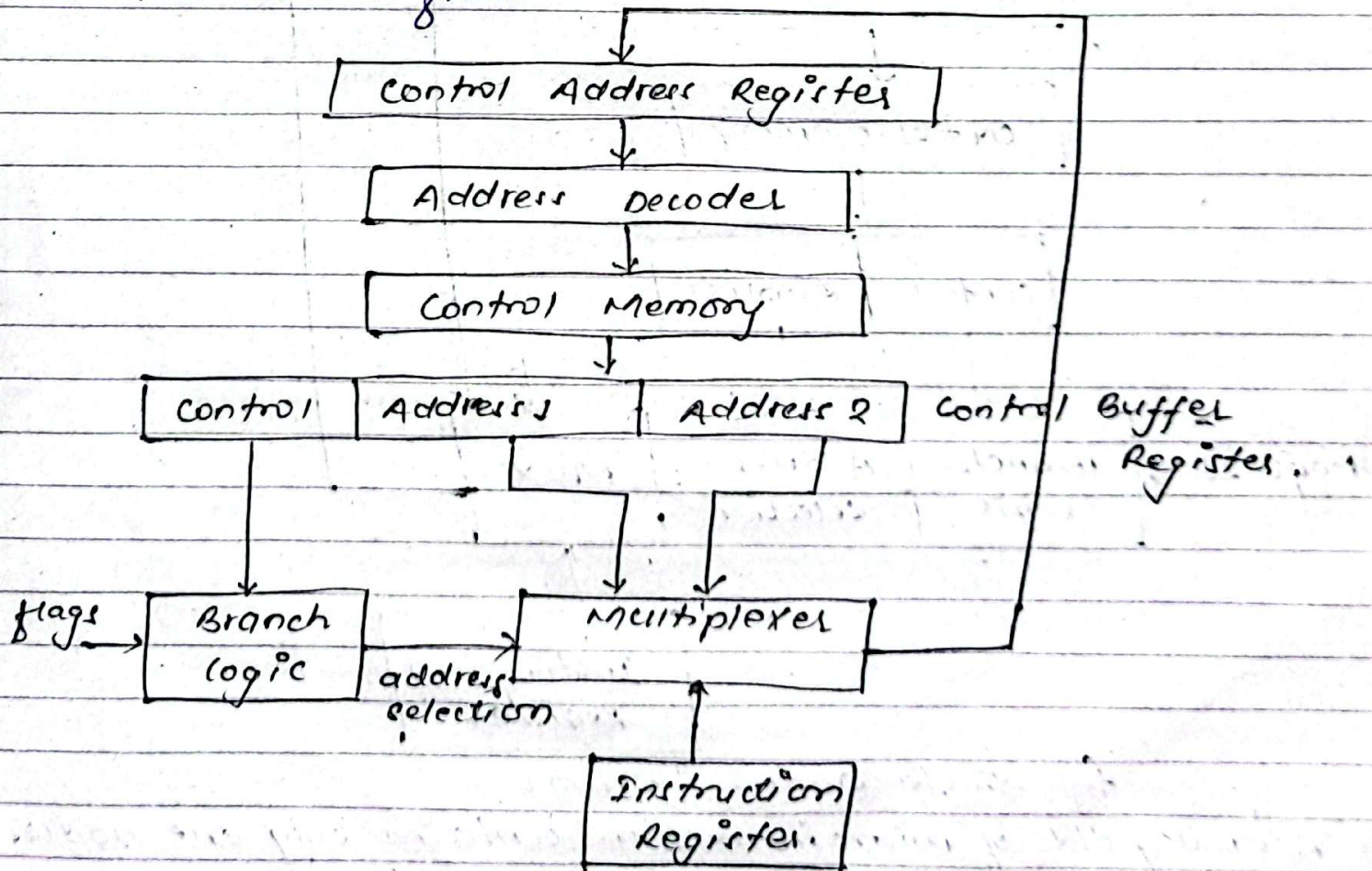


fig: two address field format.

↳ Address field of microinstruction contains two separate control memory addresses and MUX selects only one.

↳ Branch logic generates mux selection input by evaluating flags.

6. Single Address format

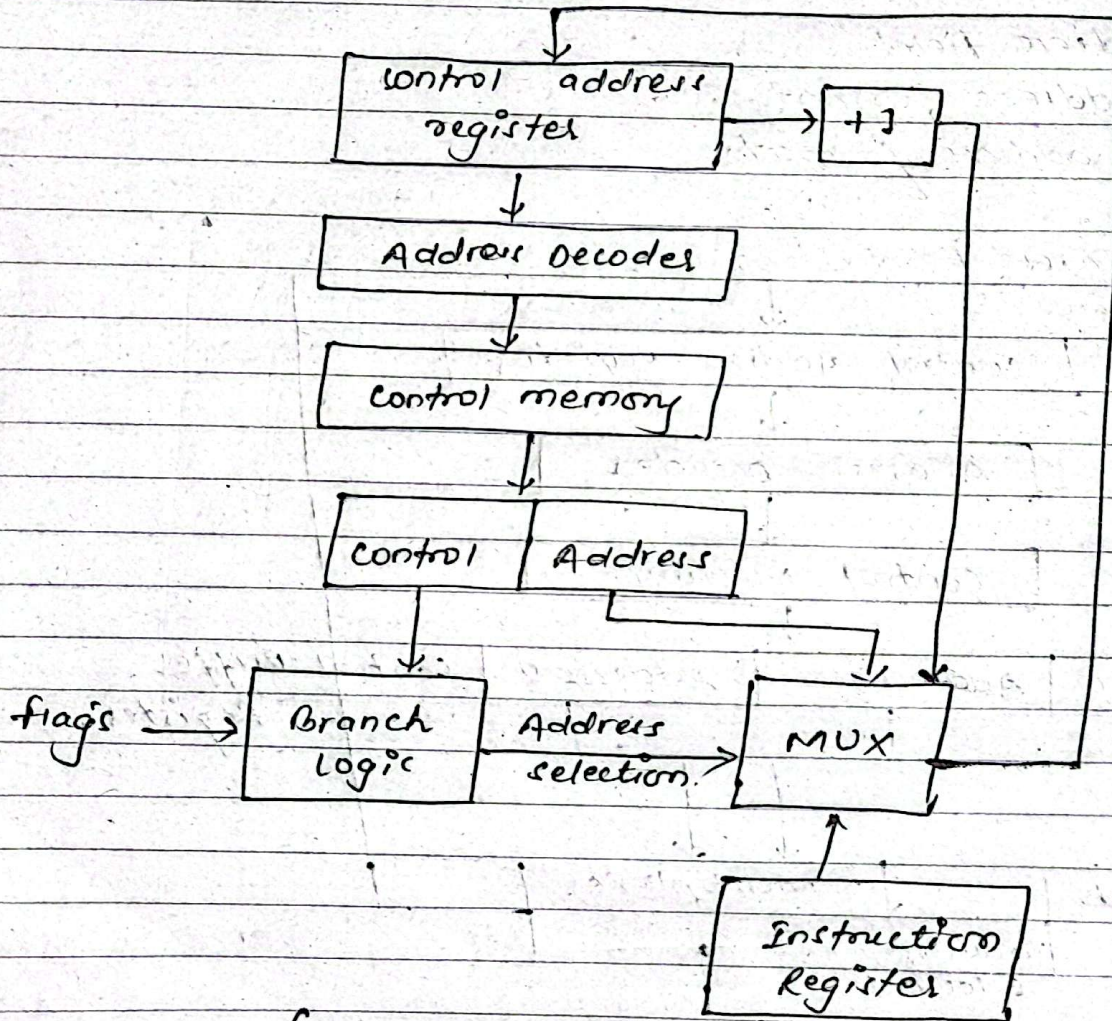


fig: Single address format

↳ Address field of microinstruction contains only one address field.

↳ MUX selects the microinstruction address field or incremented control address register.

3. variable format

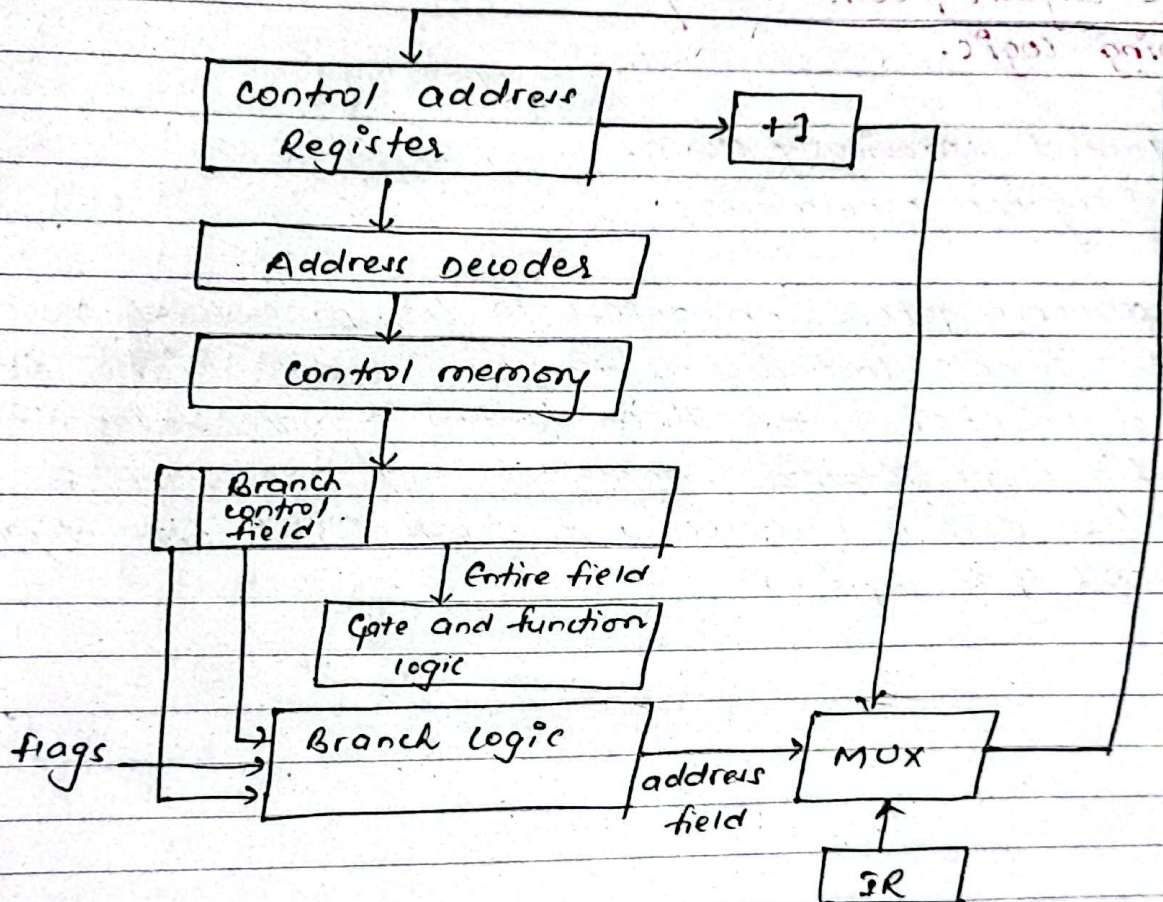


fig: variable format sequencing.

- ↳ In this technique two formats are used. In such a technique, one bit is needed in the microinstruction to differentiate between control microinstruction or a branching microinstruction.
- ↳ The first format provides the control microinstruction, while the second format provides the branch logic and address.
- ↳ In this approach, an extra cycle is needed for branch microinstruction.

5.3 Basic Layout:

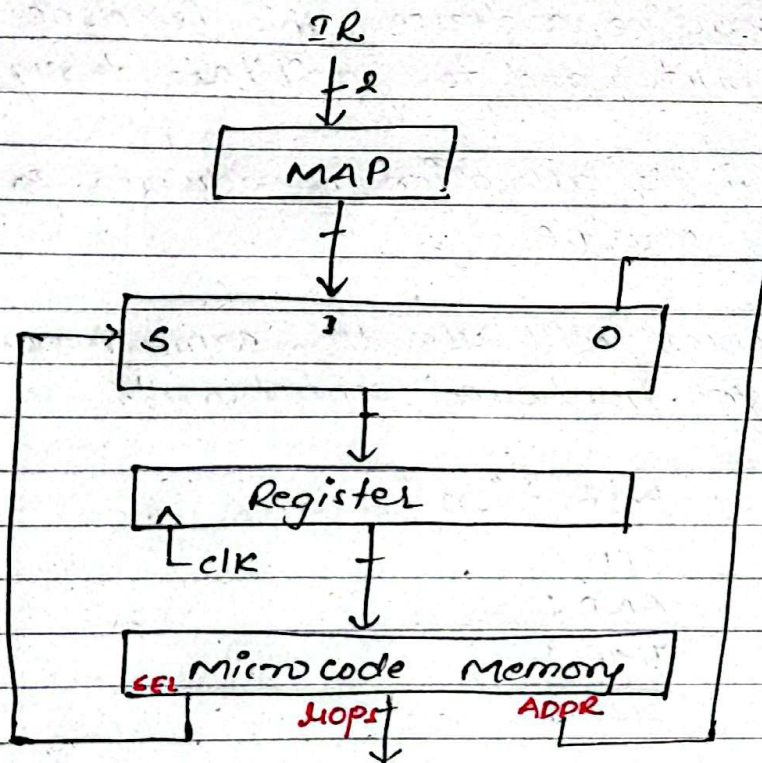


fig: microsequences for the very simple CPU.

- ↳ only two possible addresses are used: opcode mapping and an absolute jump.
- ↳ As there are two possible addresses, the microsequences must select one or other so, MUX is used.
- ↳ The select bit is generated by the microsequences to choose the correct next address.
- ↳ There are total of nine states, each of which is represented by microinstruction so the minimum number of bits needed to select among them is 4. Thus, the output is also 4 bits wide.

Control sequence generation and design of mapping logic:

- ↳ first assign each state of the finite state machine to an address in microcode.

- ↳ It is not necessary to assign states within the fetch or execute routine to consecutive addresses. This is done to improve microcode readability and to facilitate debugging.
- ↳ The primary consideration is the allocation of addresses for the first state of each execute routine.
- ↳ For the CPU, the microsequences will use the same mapping function that was used for hardwired control unit:

Address	Instruction
1000	ADD 1
1010	AND 1
1100	JMP 1
1110	INC 1

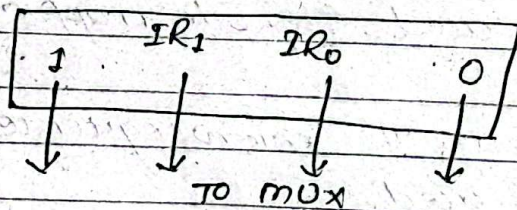


fig: mapping logic for the very simple microsequence.

- ↳ We assigned consecutive states to consecutive locations in microcode memory. The resulting assignment is shown below:

State	Address
fetch 1	0000 (0)
fetch 2	0001 (1)
fetch 3	0010 (2)

ADD 1	1000 (8)
ADD 2	1001 (9)
AND 1	1010 (10)
AND 2	1011 (11)
JMP 1	1100 (12)

INC 1 1110 (13)

Table: state addresses for the very simple microsequence.

Now set up the microcode to sequence through these states properly.

State	Address	SEL	ADDR
fetch 1	0000	0	0001
fetch 2	0001	0	0010
fetch 3	0010	1	XXXX
ADD 1	1000	0	1001
ADD 2	1001	0	0000
AND 1	1010	0	1011
AND 2	1011	0	0000
JMP 1	1100	0	0000
INC 1	1110	0	0000

Table: partial microcode for the very simple microsequence

To go unconditionally to a specific state, the microsequence supplies the address of that state via the ADDR field and selects it using the select field.

To go from fetch 1 to fetch 2, the microsequence will specify SEL=0 and ADDR=0001 in location 0, the location corresponds to fetch 1.

↳ setting $SEL=0$ causes the microsequence to get its next address from the ADDR field; setting the ADDR field to 0001 causes it to go to the location corresponding to state fetch 2.

6.4. Generation of micro operation using Horizontal microcode.

↳ Each μop is represented by one bit in each instruction. The greater the number of μops , the larger each word of $\mu code$ will be.

Mnemonics

ARPC

AR DR

PC IN

PC DR

DRM

IR DR

PLUS

AND

AC IN

μops .

$AR \leftarrow PC$

$AR \leftarrow DR [5 \dots 0]$

$PC \leftarrow PC + 1$

$PC \leftarrow DR [5 \dots 0]$

$DR \leftarrow M$

$IR \leftarrow DR [7, 6]$

$AC \leftarrow AC + DR$

$AC \leftarrow AC \wedge DR$

$AC \leftarrow AC + 1$

Table: μops and their mnemonics for the very simple CPU.

state	Address	S E	A P C	A R R	P C N	P C R	D R M	I R R	P L Y	A N D	A C N	ADDR.
fetch 1	0000 (0)	0	1	0	0	0	0	0	0	0	0	0001
fetch 2	0001 (1)	0	0	0	1	0	1	0	0	0	0	0010
fetch 3	0010 (2)	1	0	1	0	0	0	1	0	0	0	XXXX
ADD 1	1000 (8)	0	0	0	0	0	1	0	0	0	0	1001
ADD 2	1001 (9)	0	0	0	0	0	0	0	1	0	0	0000
AND 1	1010 (10)	0	0	0	0	0	1	0	0	0	0	1011
AND 2	1011 (11)	0	0	0	0	0	0	0	0	1	0	0000
JMP 1	1100 (12)	0	0	0	0	1	0	0	0	0	0	0000
INC 1	1110 (14)	0	0	0	0	0	0	0	0	0	1	0000

Table: Preliminary horizontal microcode for the very simple microsequences.

• optimizing control signals.

↳ for all states, ARDR and IRDR have same value. we don't need two ops to represent the same value so, merging both ops i.e. $AR \leftarrow DR [5..0]$ and $IR \leftarrow DR [7..6]$ we get optimized horizontal code.

state	Address	S E	A P C	A R R	P C N	P C R	D R M	P L Y	A N D	A C N	ADDR
fetch 1	0000	0	1	0	0	0	0	0	0	0	0001
fetch 2	0001	0	0	0	1	0	1	0	0	0	0010
fetch 3	0010	1	0	1	0	0	0	0	0	0	XXXX
ADD 1	1000	0	0	0	0	0	1	0	0	0	1001
ADD 2	1001	0	0	0	0	0	0	1	0	0	0000
AND 1	1010	0	0	0	0	0	1	0	0	0	1011
AND 2	1011	0	0	0	0	0	0	0	1	0	0000
JMP 1	1100	0	0	0	0	1	0	0	0	0	0000
INC 1	1110	0	0	0	0	0	0	0	0	1	0000

Generating the micro-operations using Vertical Microcodes.

- ↳ In Horizontal microcodes, over 85% of the bits are inactive i.e. consists zero.
- ↳ Vertical microcode offers a way to reduce the number of bits in this section.
- ↳ In vertical microcode, the micro-operations are grouped into fields such that no more than one micro-operation in a field is active during any state.
- ↳ For example, a field with 8 different micro-operations would require 3 bits; each value from 000 to 111.
- ↳ The micro-operation field bits are off from the microcode memory to a decoder. The outputs of the decoder are the micro-operations ~~direct~~ generated under horizontal microcode.
- ↳ Whenever two micro-operations occur during the same state, assign them to different fields.
- ↳ Include a NOP in each field if necessary. Distribute the remaining micro-operations to make best use of the micro-operation field bits.
- ↳ Group together micro-operations that 'modify' the same register in the same field.

M1	M2
NOP	NOP
DECR	PCIN
ADD	PCDR
PLUS	ARPC
AND	AIDR

- ↳ Each field has 5 micro-operations, thus requiring 3 bits per field, 6 bits in total.

23872

↳ Arrange it again

M ₁		M ₂	
values	M-ops	values	M-ops
000	NOP	0	NOP
001	DRM	1	PCIN
010	ARPC		
011	ADDR		
100	PCDR		
101	PLUS		
110	AND		
111	ACIN		

Table: M-op field assignment and values

↳ This combination requires 4 bits and is the best possible combination we can read.

State	Address	SEL	M ₁	M ₂	ADDR
fetch ₁	0000 (0)	0	010	0	0001
fetch ₂	0001 (1)	0	001	1	0010
fetch ₃	0010 (2)	1	011	0	xxxx
ADD ₁	1000 (8)	0	001	0	1001
ADD ₂	1001 (9)	0	101	0	0000
AND ₁	1010 (10)	0	001	0	1011
AND ₂	1011 (11)	0	110	0	0000
JMP ₁	1100 (12)	0	100	0	0000
INC ₁	1110 (14)	0	111	0	0000

Table: vertical microcode for the very simple microsequence.

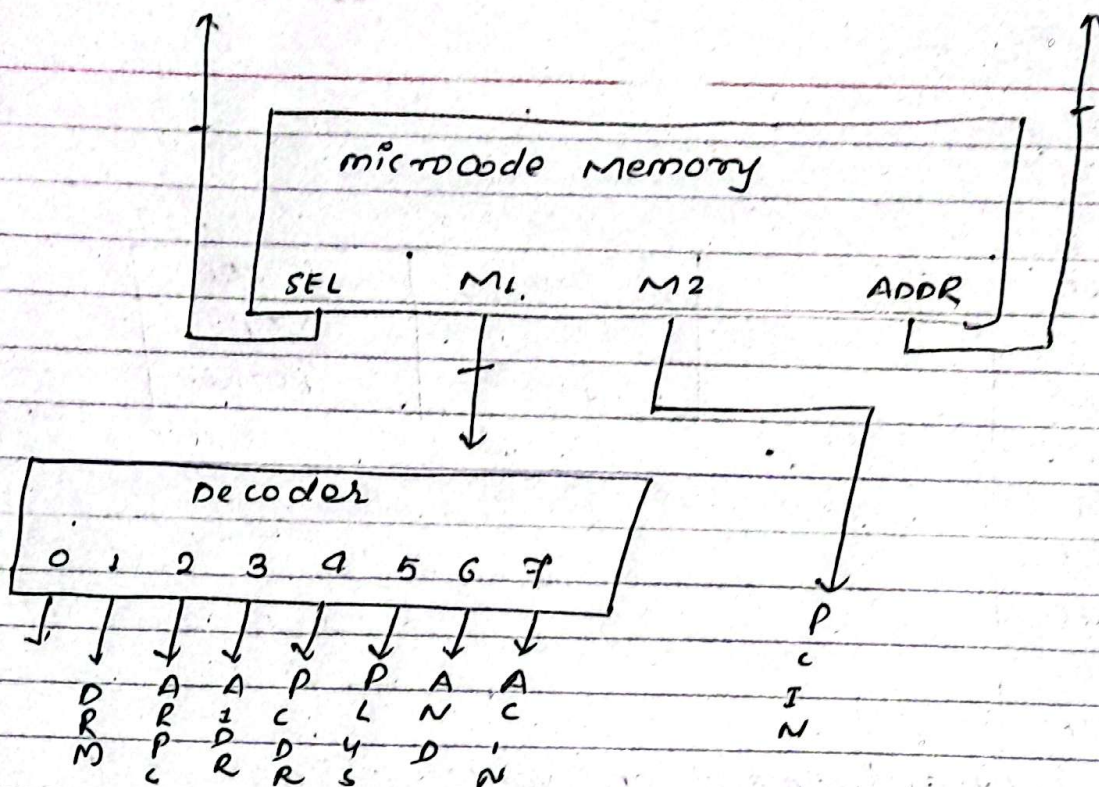


fig: Generating uops from vertical ucode for the very simple cpu.

5.7

#.

Hardwired Control unit

1. Hardwired control unit generates the control signals needed for the processor using logic circuits.

2. It is difficult to modify as the control signals that need to be generated are hard wired.

3. Only limited number of instructions are used due to the hardware implementation.

4. It cannot handle complex instructions as the circuit design for it becomes complex.

5. Used in computer that makes use of Reduced Instruction set Computer (RISC).

6. Fast in operation execution.

7. Difficult to debug and test.

Microprogrammed C.U.

1. Microprogrammed control unit generates the control signals with the help of micro instructions stored in control memory.

2. Easy to modify as the modification need to be done only at instruction level.

3. Control signals for many instructions can be generated.

4. It can handle complex instructions.

5. Used in computer that make use of Complex instruction set Computers (CISC).

6. Slower operation due to micro code decoding.

7. Easier to debug and test.

#. Reducing the number of Microinstruction:

There are two ways to take advantage of this redundancy to reduce the number of microinstructions needed by the control unit. The first method by ~~the~~ uses microsubroutines to combine repeated microoperations into single block of microinstructions, which are accessed by two or more execute routines. The other method uses microcode jump to access microinstruction shared by two or more routines.

1. Microsubroutine

A microsubroutine is a sequence of microinstructions that are grouped together to perform a specific task. for eg: there might be microsubroutine for adding two numbers, a microsubroutine for subtracting two numbers and so on.

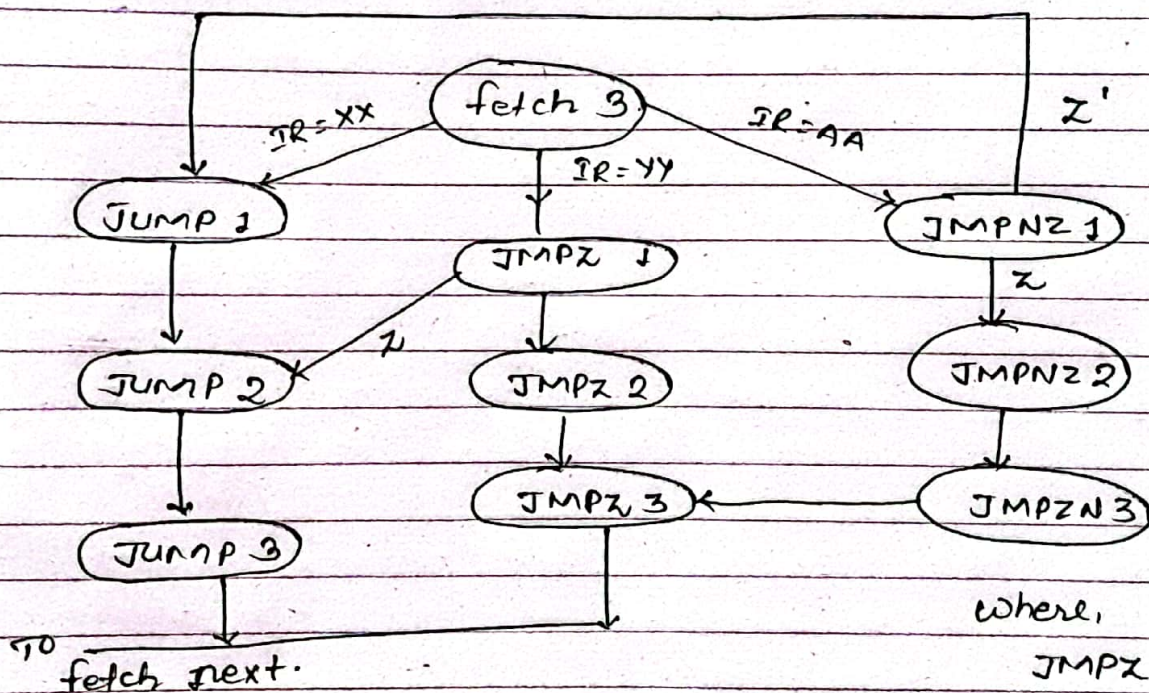
microsubroutine can help to reduce the number of μ -instructions in a few ways. first, they can be used to combine repeated micro-operations into a single block of micro-instructions. for example, the microsubroutine for adding two numbers might contain the microoperations for loading the two numbers, adding them together, and storing the result. This would eliminate the need to have separate μ -instructions for each of these μ -operations.

In similar.

2. microcode Jump

Microcode jump is a technique used to reduce the number of μ -instructions in a computer program. It works by allowing the control unit to jump directly to a specific μ -instruction, rather than having to execute all of the μ -instructions in sequence.

This can be useful in situations where there are a number of microinstructions that are executed repeatedly, such as the microinstructions for performing a jump or a conditional branch.



where,

JMPZ = Jump if Zero

JMPNZ = Jump if not zero

↳ Z and Z' are the control signals

↳ IR=XX

IR=YY

IR=AA are the random value cases.